

Google Cloud Platform - Day 3 Tutorial

Learn Pub/Sub, Load Balancing, and App Engine (Beginner-Friendly)

1 Pub/Sub

2 Load Balancer

3 App Engine

4 Quiz

Welcome to Day 3!

Today, we'll learn three powerful Google Cloud tools that help you build fast, reliable applications. Don't worry if you're not a programmer - we'll explain everything in simple terms!

Think of it like a restaurant: Today you'll learn how to take orders (Pub/Sub), manage the line of customers (Load Balancer), and run the kitchen (App Engine). Each tool does one job really well!

What You'll Learn Today



Pub/Sub (Publish/Subscribe)

A message system that lets applications talk to each other



Internal Load Balancer

Spreads work across multiple servers so nothing gets overloaded



App Engine

A place to run your programs without managing servers

Part 1: Google Cloud Pub/Sub

Time: 1.5 hours | **Difficulty:** ☆ Easy

What is Pub/Sub?

Pub/Sub stands for "Publish/Subscribe". Think of it like a newspaper:

Publishers

(like the newspaper company) create content and send it out

Topics

(like different sections - sports, news, weather) organize the content

Subscribers

(like you) choose which sections you want to read

Real example: Imagine an online store. When someone places an order, that event gets published to a topic. Then:

- The payment system subscribes and processes the payment
- The inventory system subscribes and updates stock
- The email system subscribes and sends a confirmation

All three services work independently, but all know about the order!

🔗 Lab 1: Create a Topic and Send Messages

Let's create a message topic and send some test messages. No coding needed - we'll use the Google Cloud Console (a website).

Step 1: Open Google Cloud Console

Go to

<https://console.cloud.google.com>

Sign in with your Google account

Make sure you're in the correct project (see top of page)

 You should see the Google Cloud main dashboard with various service options

Step 2: Enable Pub/Sub API

First, we need to tell Google Cloud we want to use Pub/Sub.

Click

the search bar at the top (magnifying glass icon)

Type

"Pub/Sub API" and press Enter

Click

on "Pub/Sub API" in the results

Click

the large blue

ENABLE

button

Wait

30 seconds for it to finish (you'll see a loading message)

 **Tip:** Enabling APIs is like asking Google Cloud permission to use a tool. You only need to do this once per project.

Step 3: Create a Topic

Now let's create our first topic to send messages.

Click

on "Pub/Sub" in the left menu (or search for "Pub/Sub")

Look for

the blue

CREATE TOPIC

button and click it

In the dialog box, type:

my-first-topic

Click

the

CREATE TOPIC

button

 The dialog box will ask for a topic name. Enter "my-first-topic"

✔ **Congratulations!**

You've created your first topic! You should see it listed under "Topics"

Step 4: Create a Subscription

A topic needs subscribers to receive messages. Let's create one.

Click

on your

my-first-topic

to open it

Look for

the

Subscriptions

tab or button

Click

CREATE SUBSCRIPTION

(blue button)

Type this name:

my-first-subscription

Leave other settings as default

(don't change anything else)

Click

CREATE

✔ **Great!**

Now messages sent to the topic will be available for this subscription to receive.

Step 5: Send a Test Message

Let's send a message and watch it arrive!

Go back to

your topic (click the topic name)

Click

the

PUBLISH MESSAGE

button (near the top)

In the Message box, type:

```
Hello! This is my first Pub/Sub message! 🍷
```

Click

PUBLISH

📷 You should see "Message published successfully"

Step 6: Receive the Message

Let's receive the message with our subscription.

Click

on the

Subscriptions

tab in Pub/Sub

Click

on your

my-first-subscription

Click

the

PULL

button at the top

Click

PULL MESSAGES

in the dialog

📺 You should see your message appear! It will show the message you just published.

✓ Excellent Work!

You've just sent and received a message! That's how Pub/Sub works.

🔗 Lab 2: Test Real-World Scenarios

Scenario: Order Notifications

Let's imagine we're running an online store. Create a new topic for order events.

Create a new topic:

order-events

Create a subscription:

payment-processor

Create another subscription:

email-sender

Publish this message to the topic:

```
{"orderId": "12345", "amount": 99.99, "customerEmail": "john@example.com"}
```

Now both subscriptions can receive the same message! Try pulling from each subscription and you'll see they both get it.

Why is this powerful? The payment processor doesn't know about the email system, and the email system doesn't know about the payment processor. They're completely separate. But when an order is created, both automatically get notified. This is called "decoupling" and it makes systems more flexible.

📋 Pub/Sub Checklist

Did you complete all these?

- ☐ Enabled the Pub/Sub API
- ☐ Created at least 2 topics
- ☐ Created at least 2 subscriptions
- ☐ Published messages to a topic

☐ Received messages from a subscription



Want to see it in action?

[Watch a Pub/Sub Tutorial Video →](#)

Part 2: Internal Load Balancer

Time: 1.5 hours | Difficulty: ☆☆ Medium

What is a Load Balancer?

Imagine a busy restaurant. If all customers go to one cashier, there's a long line. But if you have multiple cashiers and split customers between them, everyone gets served faster!

A load balancer does the same thing for computers:

- When a request comes in, instead of sending it to ONE server
- It sends it to one of MANY servers
- This spreads the work around, so nothing gets overloaded
- It also checks if a server is broken, and doesn't send requests to it



Internal Load Balancer

A special load balancer that works INSIDE your private network (not on the public internet)

↩ Lab 3: Create Your First Load Balancer

We'll create a simple load balancer that spreads traffic across multiple servers.

Step 1: Create a Network

First, we need to create a private network for our servers.

Search for

"VPC Network" in Google Cloud Console

Click

on "VPC Networks"

Click

the blue

CREATE VPC NETWORK

button

Fill in the form:

- **Name:**
my-network
- **Subnet creation mode:**
Custom

Scroll down and click

ADD SUBNET

Fill in the subnet:

- **Name:**
my-subnet
- **Region:**
us-central1 (or closest to you)
- **IPv4 range:**
10.0.0.0/20

Click

the blue

CREATE

button at the bottom

i What's happening? We're creating a private network (like creating a private hallway) where servers can talk to each other safely.

Step 2: Create Servers

Now let's create some servers that will handle traffic.

Search for

"Compute Engine" or "Instances"

Click

on "Compute Engine"

Click

CREATE INSTANCE

Fill in the details:

- **Name:**
server-1
- **Region:**
us-central1
- **Zone:**
us-central1-a
- **Machine type:**
e2-micro (smallest, cheapest option)

Scroll down and click

"CHANGE" next to Firewall (if available)

Leave other settings as default

Click

CREATE

Repeat this process to create

server-2 and server-3

🔗 **Creating multiple servers:** Just repeat the above steps but change the name to server-2, then server-3. They'll all be in the same network.

Step 3: Create a Load Balancer

Now let's create the load balancer to distribute traffic.

Search for

"Load Balancing"

Click

on "Load Balancing"

Click

the blue

CREATE LOAD BALANCER

button

Choose "Internal Load Balancer"

(NOT the external one)

Click

CONFIGURE

i Internal vs External: Internal = only servers in your network can reach it. External = anyone on the internet can reach it. We want Internal for this exercise.

Step 4: Configure Load Balancer

Name:

my-load-balancer

Region:

us-central1

Network:

my-network

Subnet:

my-subnet

Click

NEXT STEP

Step 5: Add Backend Servers

Tell the load balancer which servers to send traffic to.

Click

CREATE NEW BACKEND SERVICE

Name:

my-backend

Protocol:

TCP

Named port:

http with port 80

Scroll down and select your instances

(server-1, server-2, server-3)

Click

CREATE

✓ **Load Balancer Created!**

You've successfully created an internal load balancer. It will now spread traffic between your servers.

📋 **Load Balancer Checklist**

Did you complete all these?

- ☐ Created a VPC network
- ☐ Created a subnet in that network
- ☐ Created 3 server instances
- ☐ Created an internal load balancer
- ☐ Connected the servers to the load balancer

Part 3: App Engine

Time: 1.5 hours | Difficulty: ☆☆ Medium

What is App Engine?

App Engine is like a restaurant kitchen that cooks food when you order it. You don't own the kitchen, and you don't need to hire staff - you just tell Google Cloud what to cook, and it does it!

Traditional way (you manage servers): You buy computers, set them up, keep them running, fix them when they break, upgrade them, etc. (Like owning a restaurant kitchen)

App Engine way (Google manages servers): You just upload your program, and Google runs it for you. They handle everything else. (Like ordering takeout)

🔗 Lab 4: Deploy a Simple Application

Let's deploy a real application to App Engine. Don't worry - we'll do it step by step!

Step 1: Prepare Your Computer

We need to install two tools.

A. Install Google Cloud CLI

Go to:

<https://cloud.google.com/sdk/docs/install>

Choose your computer type:

Windows, Mac, or Linux

Download and run the installer

Follow the installation steps

Open Terminal/Command Prompt and type:

```
gcloud --version
```

You should see a version number.

If you do, installation worked!

B. Install Go (Programming Language)

Go to:

<https://golang.org/dl/>

Download the latest version for your computer

Run the installer and follow steps

Open Terminal/Command Prompt and type:

```
go version
```

You should see a version number. Installation worked!

🔗 **Why these tools?** Google Cloud CLI lets you talk to Google Cloud from your computer. Go is a simple programming language we'll use to write a small application.

Step 2: Create Your Application Folder

Open Terminal or Command Prompt

Create a new folder:

```
mkdir my-app cd my-app
```

Step 3: Create Your Program File

We'll create a simple Go program. Don't worry about understanding every line - this is just to show how it works.

Create a file named

main.go (you can use Notepad, VS Code, or any text editor)

Copy and paste this code:

```
package main import ( "fmt" "log" "net/http" "os" ) func helloHandler(w http.ResponseWriter, r *http.Request) {
fmt.Fprint(w, "Hello! Welcome to my App Engine application! 🍷") } func main() { http.HandleFunc("/", helloHandler)
port := os.Getenv("PORT") if port == "" { port = "8080" } log.Printf("Server starting on port %s", port)
log.Fatal(http.ListenAndServe(fmt.Sprintf(":%s", port), nil)) }
```

Save the file

in your my-app folder

What does this program do? It's a simple web server. When someone visits it, it says "Hello! Welcome to my App Engine application!"

Step 4: Create Configuration File

We need to tell App Engine how to run our program.

Create a new file named

app.yaml in your my-app folder

Copy and paste this:

```
runtime: go119 env: standard
```

Save the file

What is this? This tells Google Cloud: "This is a Go program, using Go version 1.19"

Step 5: Test Your Program Locally

Before uploading to Google Cloud, let's test it on your computer.

In Terminal/Command Prompt, make sure you're in the my-app folder

Type:

```
go run main.go
```

You should see:

```
Server starting on port 8080
```

Open your web browser

Go to:

<http://localhost:8080>

You should see:

"Hello! Welcome to my App Engine application! 🍹"

✓ **It Works!**

Your program is running on your computer!

Stop the program by pressing

Ctrl+C in Terminal

Step 6: Deploy to App Engine

Now let's upload your program to Google Cloud.

In Terminal, make sure you're logged into Google Cloud:

```
gcloud auth login
```

Set your project ID (replace YOUR-PROJECT-ID with your actual project ID):

```
gcloud config set project YOUR-PROJECT-ID
```

Deploy your app (run this from your my-app folder):

```
gcloud app deploy
```

If asked, choose region:

us-central (or nearest to you)

Type "y" when asked for confirmation

Wait for it to finish (this takes a few minutes)

👁️ You'll see lots of text as Google Cloud builds and deploys your app. Look for the final line with your application URL!

Step 7: View Your Live Application

Your app is now live on the internet!

In Terminal, type:

```
gcloud app browse
```

This will open your application in a browser

You should see:

"Hello! Welcome to my App Engine application! 🐼"

🎉 **Congratulations!**

Your application is now live on the internet! You can share the URL with anyone and they can see it.

Your URL looks like: <https://YOUR-PROJECT-ID.appspot.com/>

Step 8: View Your Application in Google Cloud Console

[Go to Google Cloud Console](#)

[Search for "App Engine"](#)

[Click on "App Engine"](#)

[You'll see your application listed](#)

[Click on it to see statistics](#)

What's happening? Google Cloud is:

- Running your program on multiple servers
- Automatically scaling up when traffic increases
- Automatically scaling down when traffic decreases
- Keeping your app running 24/7
- You don't have to do anything!

App Engine Checklist

Did you complete all these?

- ☐ Installed Google Cloud CLI
- ☐ Installed Go
- ☐ Created main.go file
- ☐ Created app.yaml file
- ☐ Tested program locally
- ☐ Deployed to App Engine
- ☐ Accessed your live application
- ☐ Viewed it in Google Cloud Console

Glossary - Simple Definitions

[API \(Application Programming Interface\)](#)

A way for programs to talk to each other. Think of it like a menu at a restaurant - you make a request (order) and get a response (food).

Deployment

Publishing or uploading your program to a server so other people can use it.

Instance

A single server or computer. Like one cashier in a restaurant.

Load Balancer

A tool that distributes work between multiple servers so none gets overwhelmed.

Message

Information sent from one program to another through Pub/Sub.

Network

A private connection between servers. Like a private hallway.

Publisher

A program that sends messages to a topic in Pub/Sub.

Pub/Sub

Short for Publish/Subscribe. A messaging system where programs can send and receive messages.

Serverless

You run programs without managing servers. Google Cloud handles the servers for you.

Subscription

A connection to a topic in Pub/Sub that allows you to receive messages.

Topic

A place where messages are sent in Pub/Sub. Like a newspaper section.

Traffic

The flow of requests to your application. Like customers flowing through a store.

?Frequently Asked Questions

Why do I need Pub/Sub if I can just call another program directly?



Does a Load Balancer make my app faster?



Does App Engine cost money?



What if my App Engine application crashes?



Can I see logs of what happened?



How do I know if my Load Balancer is working?



✓ Great Job! You Did It!

What You Learned Today



Pub/Sub

How to send messages between programs without them talking directly



Load Balancer

How to spread traffic across multiple servers



App Engine

How to deploy a real application to the internet

Key Takeaways

Pub/Sub

lets programs talk asynchronously (at different times)

Load Balancers

distribute work so everything stays fast

App Engine

lets you focus on your program, not servers

Google Cloud handles the hard stuff - scaling, reliability, monitoring

Next Steps

If You Want to Learn More

1. **Add a database** to your App Engine app (Cloud SQL or Firestore)
2. **Connect Pub/Sub to your App Engine app** - make it publish messages
3. **Create a subscriber service** - receive messages from Pub/Sub
4. **Deploy multiple versions** of your app and test them
5. **Set up monitoring** to watch your app's performance

Share Your Learning 🌟

Mark Complete 🏆

Helpful Resources

Official Google Cloud Documentation

Pub/Sub:

<https://cloud.google.com/pubsub/docs>

Load Balancing:

<https://cloud.google.com/load-balancing/docs>

App Engine:

<https://cloud.google.com/appengine/docs>

Video Tutorials

Google Cloud Pub/Sub Overview:

[YouTube Search →](#)

Load Balancing Tutorial:

[YouTube Search →](#)

App Engine Tutorial:

[YouTube Search →](#)

Free Trial Credits

If you signed up for Google Cloud, you should have \$300 in free credits. This is enough to experiment with these services without spending money!

Get Help

Google Cloud Console has "SUPPORT"

- click it for live chat help

Stack Overflow:

Search your question - someone has probably answered it

Google Cloud Community:

Ask questions, get help from experts

🎉 You've completed Google Cloud Platform Day 3!

Now you understand the basics of Pub/Sub, Load Balancing, and App Engine.

Ready for Day 4? You'll learn about databases and storage!